NALA — Senior Analytics Engineer Technical Assessment

About NALA

NALA is building payments for the next billion. We enable seamless cross-border payments from the UK, US, and EU to Africa and Asia. We are a fast-growing fintech backed by world-class investors, operating at the intersection of technology, finance, and impact.

The Role

As Senior Analytics Engineer, you would own NALA's data transformation layer. Your work will ensure that every model is documented, tested, and structured in a way that both humans and AI agents can interpret and trust. This is a foundational engineering role — your primary remit is dbt refactoring, testing, documentation, performance optimisation, and cost reduction.

Assessment Brief

You are being given the schemas of three data sources that we wish to land in NALA's Snowflake warehouse, along with a set of business requirements. Your task is to design and build a dbt project from scratch that transforms this raw data into a governed, consumption-ready data layer.

You may use any tools you like, including AI assistants. We expect you to — this is an AI-native team. We ask that you log all prompts used (see Deliverables).

Time budget: 2-3 hours of focused work. You have 5 days to submit.

Data Platform Context

Component	Technology
Data warehouse	Snowflake

Component	Technology
Transformation	dbt (dbt-snowflake)
Semantic layer	dbt Semantic Layer (MetricFlow)
BI tool	Hex
Orchestration	To be determined (you should propose an approach)

How Data Reaches Snowflake

- **Sources 1 and 2** are PostgreSQL databases. Data is replicated into Snowflake using **StreamServe**, a CDC (Change Data Capture) connector. StreamServe replicates row-level changes from Postgres into Snowflake in near-real-time.
- **Source 3** is loaded via **Fivetran** as a daily batch sync. Fivetran manages the connection and loads data into a raw schema in Snowflake once per day at approximately 06:00 UTC.

All raw data lands in the `RAW` database in Snowflake.

Source 1 — Payments Backend

Database: PostgreSQL (application backend)
Replication: StreamServe CDC to Snowflake
Description: The core transactional database powering NALA's consumer remittance product. Contains user records, transaction lifecycle data, disbursement routing, operational tasks, and FX rate history.

Table: `users`

The primary user table. One row per registered user.

Column	Type	Description
id	uuid	Primary key. Unique identifier for the user.
status	string	Current account status. Values: `ACTIVE`, `INACTIVE`, `SUSPENDED`, `PENDING`.
source	json	Object describing how the user was acquired. Contains a `link` key with the referral or attribution URL.
sender_country	string	ISO country code for the country the user sends money from (e.g. `GB`, `US`, `DE`).
created	timestamp	Timestamp when the user registered.

Column	Type	Description
allowed	timestamp	Timestamp when the user was approved to transact (post-KYC). Null if not yet approved.
client_properties	json	Device and client metadata set by the mobile app.
used_invitation_code_id	uuid	Foreign key to the invitation code used at signup. Null if no code was used.
profile_picture	string	URL to the user's profile image. Null if not set.
usage	json	Object containing user usage preferences. Contains `sender_country` and `recipient_countries` keys.

Approximate row count: 2,000,000
Update frequency: Rows are updated within seconds of a change in the application (e.g. status change, profile update).

Table: `users_account`

Account records. Each user has one or more accounts (e.g. personal account, business account). An account is the entity that owns transactions.

Column	Type	Description
id	uuid	Primary key. Unique account identifier.
owner_id	uuid	Foreign key to `users.id`. The user who owns this account.
name	string	Display name for the account.
type	string	Account type. Values: `PERSONAL`, `BUSINESS`.
status	string	Account status. Values: `ACTIVE`, `INACTIVE`, `SUSPENDED`.
features	json	Feature flags and capabilities enabled for this account.
created	timestamp	Timestamp when the account was created.
last_updated	timestamp	Timestamp of the most recent update to this record.
profile_id	uuid	Reference to the account's profile configuration.
limit_level	string	The transaction limit tier assigned to this account.
client_properties	json	Client-side metadata.
deleted	timestamp	Timestamp when the account was soft-deleted by the application. Null if the account is active.

Approximate row count: 2,200,000
Update frequency: Updates within seconds of application changes.

Table: `wallets`

User wallets. Each account can have one or more wallets in different currencies. Wallets hold balances and are the source or destination of funds in transactions.

Column	Type	Description
id	uuid	Primary key. Unique wallet identifier.
account_id	uuid	Foreign key to `users_account.id`. The account that owns this wallet.
type	string	Wallet type. Values: `CASH`, `CREDIT`.
currency_code	string	ISO currency code (e.g. `USD`, `GBP`, `EUR`, `KES`, `TZS`).
provider_name	string	The banking or wallet provider backing this wallet.
enabled	boolean	Whether the wallet is currently active and usable.
created_at	timestamp	Timestamp when the wallet was created.
updated_at	timestamp	Timestamp of the most recent update.

Approximate row count: 4,000,000
Update frequency: Updates within seconds of balance changes or wallet status changes.

Table: `transactions_transaction`

The core transaction table. One row per transaction initiated by a user. A transaction represents a cross-border money transfer.

Column	Type	Description
id	uuid	Primary key. Unique transaction identifier.
user_id	uuid	Foreign key to `users.id`. The user who initiated the transaction.
account_id	uuid	Foreign key to `users_account.id`. The account the transaction belongs to.

Column	Type	Description
type	string	Transaction type. Values: `COLLECTION`, `CONVERSION`, `DISBURSEMENT`, `COLLECTION_CONVERSION`, `CONVERSION_DISBURSEMENT`, `COLLECTION_CONVERSION_DISBURSEMENT`, `EARNED_REWARD`, `REVERSAL`, `INCOMING`, `INCOMING_PEER_TO_PEER`, `OUTGOING_PEER_TO_PEER`, `CONVERSION_OUTGOING_PEER_TO_PEER`, `INCOMING_COLLECTION`.
state	string	Current transaction state. Values: `CREATED`, `COLLECTION_IN_PROGRESS`, `CONVERSION_IN_PROGRESS`, `DISBURSEMENT_IN_PROGRESS`, `WORKFLOW_IN_PROGRESS`, `WAITING_ON_WORKFLOW`, `WAITING_ON_RECIPIENT_APPROVAL`, `ON_HOLD`, `PENDING_CAPTURE`, `COMPLETED`, `FAILED`, `CANCELLED`, `EXPIRED`, `REJECTED`, `REFUNDED`, `REVERSED_TO_WALLET`, `VOIDED`.
sent_amount	decimal	The amount the sender sent, in `sent_currency`.
sent_currency	string	ISO currency code of the sending currency (e.g. `GBP`, `USD`, `EUR`).
received_amount	decimal	The amount the recipient receives, in `received_currency`.
received_currency	string	ISO currency code of the receiving currency (e.g. `KES`, `TZS`, `NGN`, `UGX`).
exchange_rate	decimal	The exchange rate applied to this transaction (1 unit of sent_currency = exchange_rate units of received_currency).
source_amount	decimal	The original amount before fees, in the sending currency.
recipient_id	uuid	Foreign key to `transactions_recipient.id`. The recipient of this transaction. Null for non-disbursement types.
recipient_account_id	uuid	Foreign key to `transactions_recipient_account.id`. The specific payment destination. Null for non-disbursement types.
workflow_id	uuid	Foreign key to the approval workflow, if the transaction requires one. Null for most transactions.
created_at	timestamp	Timestamp when the transaction was created.
updated_at	timestamp	Timestamp of the most recent state change.
expires_at	timestamp	Timestamp when the transaction offer expires if not completed.
memo	string	Optional note from the sender to the recipient.

Column	Type	Description
purpose	string	Purpose of the remittance. Values: `REMITTANCE`, `SALARY_AND_WAGES`, `GOODS_PURCHASE`, `SERVICES_PAYMENT`, `BILLS_PAYMENT`, `P2P_TRANSFER`.
from_wallet_id	uuid	Foreign key to `wallets.id`. The wallet the funds are collected from. Null for card-funded transactions.
to_wallet_id	uuid	Foreign key to `wallets.id`. The wallet the funds are disbursed to. Null for bank/mobile money payouts.
summary	json	Structured summary of the transaction. Contains nested objects: `original.source_amount.amount` (float, the pre-fee amount), `fees` (array of fee objects, each with `definition.label` and `definition.kind`).
metadata	json	Provider-specific metadata. Structure varies by payment method.
fee_label	string	Display label for the primary fee (e.g. `Transfer fee`).
fee_kind	string	Fee classification (e.g. `FIXED`, `PERCENTAGE`).

Approximate row count: 15,000,000
Update frequency: High volume. Transactions are created and updated continuously throughout the day.

Table: `transactions_collection`

Collection records. Each transaction with a collection leg (types containing `COLLECTION`) has one collection record, representing the inbound money movement — how funds are collected from the sender.

Column	Type	Description
id	uuid	Primary key. Unique collection identifier.
transaction_id	uuid	Foreign key to `transactions_transaction.id`.
account_id	uuid	Foreign key to `users_account.id`. The account being collected for.
state	string	Collection state. Values: `CREATED`, `STARTED`, `WAITING_ON_CUSTOMER`, `WAITING_ON_PROVIDER`, `COMPLETED`, `FAILED`, `CANCELLED`, `REFUNDED`, `PENDING_CAPTURE`, `CAPTURE_FAILED`, `VOIDED`.
amount	decimal	The amount being collected.

Column	Type	Description
currency	string	ISO currency code of the collection amount.
to_wallet_id	uuid	Foreign key to `wallets.id`. The wallet the collected funds are credited to.
created_at	timestamp	Timestamp when the collection was created.
updated_at	timestamp	Timestamp of the most recent state change.

Approximate row count: 14,000,000
Update frequency: High volume, updates in near-real-time as collections progress through states.

Table: `collection_payment_attempts`

Individual payment attempts within a collection. A collection may have multiple payment attempts if the first attempt fails (e.g. card declined, insufficient funds) and the user retries with a different payment method.

Column	Type	Description
id	uuid	Primary key. Unique payment attempt identifier.
collection_id	uuid	Foreign key to `transactions_collection.id`.
payment_method_type	string	The type of payment method used. Values: `CARD`, `BANK_ACCOUNT`, `APPLE_PAY`, `GOOGLE_PAY`, `IDEAL`, `SOFORT`, `WALLET`.
state	string	Attempt state. Values: `CREATED`, `STARTED`, `WAITING_ON_PROVIDER`, `COMPLETED`, `FAILED`, `CANCELLED`, `REFUNDED`, `PENDING_CAPTURE`, `CAPTURE_FAILED`, `VOIDED`.
amount	decimal	The amount of this payment attempt.
currency	string	ISO currency code.
provider_name	string	The payment provider processing this attempt.
error	text	Error message if the attempt failed. Null on success.
created_at	timestamp	Timestamp when the attempt was initiated.
updated_at	timestamp	Timestamp of the most recent state change.

Approximate row count: 16,000,000
Update frequency: High volume. Attempts are created and updated in near-real-time.

Table: `transactions_disbursement`

Disbursement records. Each transaction with a disbursement leg (types containing `DISBURSEMENT`) has one disbursement, which represents the payout leg of the money transfer. The disbursement is routed to a provider for delivery.

Column	Type	Description
id	uuid	Primary key. Unique disbursement identifier.
transaction_id	uuid	Foreign key to `transactions_transaction.id`.
state	string	Disbursement state. Values: `CREATED`, `READY`, `PREPARED`, `WAITING_ON_PROVIDER`, `COMPLETED`, `FAILED`, `REVERSED`, `AMBIGUOUS`.
amount	decimal	The amount being disbursed in the destination currency.
currency	string	ISO currency code of the disbursement amount.
recipient_id	uuid	Foreign key to `transactions_recipient.id`.
recipient_account_id	uuid	Foreign key to `transactions_recipient_account.id`. The specific payment destination.
provider_name	string	The name of the payout provider handling this disbursement (e.g. `THUNES`, `TERRAPAY`, `CELLULANT`).
created_at	timestamp	Timestamp when the disbursement was created.
updated_at	timestamp	Timestamp of the most recent state change.

Approximate row count: 12,000,000
Update frequency: High volume, updates in near-real-time as disbursements progress through states.

Table: `disbursement_attempts`

Individual payout attempts. A single disbursement may have multiple attempts if the first provider fails and the system retries with a different provider.

Column	Type	Description
id	uuid	Primary key. Unique attempt identifier.
disbursement_id	uuid	Foreign key to `transactions_disbursement.id`.
state	string	Attempt state. Values: `CREATED`, `WAITING_ON_PROVIDER`, `COMPLETED`, `FAILED`, `REVERSED`, `AMBIGUOUS`.

Column	Type	Description
provider_name	string	The payout provider used for this attempt.
provider_id	string	The provider's internal reference for this attempt.
selection_reason	string	How this provider was chosen for the attempt. Values: `PREDEFINED`, `PRIORITY_ORDER`, `SMART_ROUTING`, `PROBING`.
provider_error_category	string	Standardised classification of the provider failure reason. Null on success.
created_at	timestamp	Timestamp when the attempt was initiated.
last_updated_at	timestamp	Timestamp of the most recent state change.
eta	timestamp	Estimated time of arrival for the payout.
max_eta	timestamp	Maximum expected delivery time. Used for SLA monitoring.
error	text	Error message from the system, if the attempt failed. Null on success.
provider_error	text	Error message returned by the provider, if any. Null on success.
receipt_number	string	Provider-issued receipt or confirmation number. Null until completed.
psp_account_id	uuid	The PSP (payment service provider) account used for routing.

Approximate row count: 20,000,000

Update frequency: High volume. Attempts are created and updated in near-real-time.

Table: `transactions_recipient`

Recipient records. A recipient is a person or business that receives money from a NALA user. Recipients belong to an account and can have one or more recipient accounts (payment destinations).

Column	Type	Description
id	uuid	Primary key. Unique recipient identifier.
account_id	uuid	Foreign key to `users_account.id`. The account that created this recipient.
first_name	string	The recipient's first name.
last_name	string	The recipient's last name.

Column	Type	Description
type	string	Recipient type. Values: `INDIVIDUAL`, `BUSINESS`.
created_at	timestamp	Timestamp when the recipient was created.
updated_at	timestamp	Timestamp of the most recent update.

Approximate row count: 3,000,000
Update frequency: Moderate. New recipients are added as users set up new payees.

Table: `transactions_recipient_account`

Recipient account records. Each recipient has one or more payment destinations — the specific account, phone number, or address where money is delivered.

Column	Type	Description
id	uuid	Primary key. Unique recipient account identifier.
recipient_id	uuid	Foreign key to `transactions_recipient.id`.
type	string	Payment destination type. Values: `MOBILE_MONEY`, `BANK_ACCOUNT`, `UPI`, `BUY_GOODS`, `PAY_BILL`, `PEER_TO_PEER`.
country	string	ISO country code of the recipient's country.
currency	string	ISO currency code of the destination currency.
phone_number	string	Recipient's phone number (for mobile money). Null for non-mobile-money types.
operator	string	Mobile money operator name (e.g. `MPESA`, `AIRTEL`, `MTN`). Null for non-mobile-money types.
account_number	string	Bank account number or identifier. Null for non-bank types.
bank_code	string	Bank identifier code. Null for non-bank types.
bank_name	string	Name of the bank. Null for non-bank types.
created_at	timestamp	Timestamp when the recipient account was created.
updated_at	timestamp	Timestamp of the most recent update.

Approximate row count: 4,500,000
Update frequency: Moderate. New recipient accounts are added as users add payees.

Table: `tasks_task`

Operational tasks created by various services (fincrime, compliance, customer support). Tasks are assigned to staff for review and resolution.

Column	Type	Description
id	uuid	Primary key. Unique task identifier.
service	string	The service that created the task. Values: `fincrime`, `compliance`, `support`.
type	string	Task type, specific to the service (e.g. `user_review`, `transaction_review`, `escalation`).
state	string	Task state. Values: `OPEN`, `IN_PROGRESS`, `RESOLVED`, `CANCELLED`.
priority	string	Priority level. Values: `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.
staff_id	uuid	The staff member who created the task. Null for system-generated tasks.
assignee_id	uuid	The staff member currently assigned to the task. Null if unassigned.
associated_ids	array of strings	Array of entity IDs related to the task. Index 0 is typically the primary entity (user_id or transaction_id). Index 2 (when present) is the user_id for transaction tasks.
content	json	Task content and context. Contains keys like `disbursement_id`, `account_id`, `provider_name` depending on the task type.
created_at	timestamp	Timestamp when the task was created.
last_updated_at	timestamp	Timestamp of the most recent update.
resolution	string	Resolution outcome. Values: `APPROVED`, `REJECTED`, `ESCALATED`. Null while open.

Approximate row count: 500,000
Update frequency: Moderate. Tasks are created by automated systems and updated by staff.

Table: `fx_rates`

Historical FX rate snapshots captured from various providers. Each row is a single rate observation at a point in time.

Column	Type	Description

Column	Type	Description
id	uuid	Primary key.
source_currency	string	ISO currency code of the source (selling) currency.
destination_currency	string	ISO currency code of the destination (buying) currency.
rate	decimal	The exchange rate (1 unit of source currency = `rate` units of destination currency).
amount	decimal	The notional amount the rate was quoted for.
provider_name	string	The FX provider that supplied this rate.
created_at	timestamp	Timestamp when the rate was captured.

Approximate row count: 5,000,000
Update frequency: Rates are captured multiple times per hour for active corridors.

Source 2 — Financial Crime Service

Database: PostgreSQL (separate microservice)
Replication: StreamServe CDC to Snowflake
Description: NALA's financial crime detection system. It evaluates users and transactions against configurable rules organised into workflows. When a rule triggers, it can create operational tasks in the backend (Source 1) for manual review by the fincrime ops team.

How the System Works

1. **Rules** define individual checks (e.g. "transaction amount exceeds threshold", "user from high-risk jurisdiction"). Rules have conditions, categories, and versions.
2. **Workflows** are ordered collections of rules. A workflow defines which rules to evaluate and in what order. The `config` JSON contains the rule node graph.
3. When a business event occurs (e.g. a transaction is created, a user completes KYC), a **workflow execution** is triggered. The execution evaluates each rule in the workflow against the relevant user or transaction.
4. Each rule evaluation within a workflow execution produces a **rule execution** record with the outcome.
5. Rule executions that trigger alerts are reviewed by the fincrime ops team. These reviews are recorded in **rule execution reviews** with a determination of whether the alert was a true positive, false positive, or inconclusive.
6. Workflow executions that result in certain actions (e.g. `BLOCK_USER`, `HOLD_TRANSACTION`) trigger **tasks** in Source 1's `tasks_task` table with `service = 'fincrime'`.

Table: `rules`

Rule definitions. Each row is a version of a rule.

Column	Type	Description
id	uuid	Primary key. Unique rule version identifier.
name	string	Machine-readable rule name (e.g. `high_value_transaction_check`).
title	string	Human-readable rule title.
description	string	Detailed description of what the rule checks.
type	string	Rule type (e.g. `THRESHOLD`, `PATTERN`, `LIST_CHECK`).
category	string	Rule category (e.g. `AML`, `FRAUD`, `SANCTIONS`).
team	string	The team responsible for this rule.
condition	text	The rule's condition expression.
version	integer	Version number. Higher is newer.
created_at	timestamp	Timestamp when this version was created.
updated_at	timestamp	Timestamp of the most recent update.
deleted_at	timestamp	Timestamp when this rule version was soft-deleted. Null if active.

Approximate row count: 200
Update frequency: Rarely. Rules are updated when the fincrime team changes detection logic.

Table: `workflows`

Workflow definitions. Each workflow is an ordered collection of rules.

Column	Type	Description
id	uuid	Primary key. Unique workflow version identifier.
name	string	Machine-readable workflow name.
title	string	Human-readable workflow title.
type	string	Workflow type (e.g. `PRE_TRANSACTION`, `POST_TRANSACTION`, `USER_ONBOARDING`).
config	json	Workflow configuration. Contains a `nodes` array where each node has `type` (e.g. `RULE`) and `config.rule_id` referencing a rule.

Column	Type	Description
version	integer	Version number. Higher is newer.
created_at	timestamp	Timestamp when this version was created.
updated_at	timestamp	Timestamp of the most recent update.
deleted_at	timestamp	Timestamp when this workflow version was soft-deleted. Null if active.

Approximate row count: 50
Update frequency: Rarely. Workflows are updated when the detection strategy changes.

Table: `workflow_rules`

Mapping table linking workflows to their constituent rules.

Column	Type	Description
workflow_id	uuid	Foreign key to `workflows.id`.
rule_id	uuid	Foreign key to `rules.id`.

Primary key: Composite (`workflow_id`, `rule_id`)
Approximate row count: 300
Update frequency: Changes only when workflow compositions are updated.

Table: `workflow_executions`

Each row represents one execution of a workflow, triggered by a business event. The `context` column carries the reference back to the entity being evaluated.

Column	Type	Description
id	uuid	Primary key. Unique execution identifier.
workflow_id	uuid	Foreign key to `workflows.id`. The workflow that was executed.
result	string	Overall execution result. Values: `PASS`, `FAIL`, `ERROR`.
context	json	The business context for this execution. Contains `user_id` (string, references `Source 1 users.id`) and `transaction_id` (string, references `Source 1 transactions_transaction.id`). One or both may be present depending on the trigger event.
created_at	timestamp	Timestamp when the execution was initiated.

Column	Type	Description
started_at	timestamp	Timestamp when evaluation began.
ended_at	timestamp	Timestamp when evaluation completed.
error	text	Error details if the execution failed due to a system error. Null on success.
actions	json	Array of actions triggered by this execution (e.g. `BLOCK_USER`, `HOLD_TRANSACTION`, `CREATE_TASK`).

Approximate row count: 10,000,000

Update frequency: High volume. Executions are created continuously as transactions and user events occur.

Table: `rule_executions`

Each row represents the evaluation of a single rule within a workflow execution.

Column	Type	Description
id	uuid	Primary key. Unique rule execution identifier.
rule_id	uuid	Foreign key to `rules.id`. The rule that was evaluated.
workflow_execution_id	uuid	Foreign key to `workflow_executions.id`. The parent workflow execution.
result	string	Rule evaluation result. Values: `PASS`, `FAIL`, `ERROR`.
condition_result	string	The raw result of the condition evaluation.
context	json	Same structure as `workflow_executions.context`. Contains `user_id` and/or `transaction_id`.
created_at	timestamp	Timestamp when the rule evaluation started.
started_at	timestamp	Timestamp when condition evaluation began.
ended_at	timestamp	Timestamp when condition evaluation completed.
error	text	Error details if the rule evaluation failed. Null on success.
review	string	Review status, if the rule execution was reviewed. Values: `PENDING`, `REVIEWED`. Null if no review is required.
review_comment	text	Comment left by the reviewer. Null if not reviewed.
reviewer_id	uuid	The staff member who reviewed this execution. Null if not reviewed.

Column	Type	Description
reviewed_at	timestamp	Timestamp when the review was completed. Null if not reviewed.

Approximate row count: 30,000,000
Update frequency: Very high volume. One rule execution per rule per workflow execution.

Table: `rule_execution_reviews`

Dedicated review records for rule executions that required manual assessment. This is separate from the inline review fields on `rule_executions` and represents the formal ops review process.

Column	Type	Description
id	uuid	Primary key. Unique review identifier.
rule_execution_id	uuid	Foreign key to `rule_executions.id`.
reviewer_id	uuid	The staff member who performed the review.
review	string	Review determination. Values: `TRUE_POSITIVE`, `FALSE_POSITIVE`, `INCONCLUSIVE`.
review_comment	text	Detailed comment explaining the reviewer's determination.
reviewed_at	timestamp	Timestamp when the review was completed.

Approximate row count: 500,000
Update frequency: Moderate. Reviews are created as fincrime ops staff process alerts.

Source 3 — Amplitude Events

Platform: Amplitude (product analytics)
Replication: Fivetran daily batch sync to Snowflake
Description: Product analytics events captured from NALA's mobile app. Tracks user interactions throughout the app lifecycle — from first open through onboarding, KYC, and transactions. This data is loaded once per day via Fivetran.

Business context: The growth and product teams want to start building analytics on top of this data to understand user onboarding funnels and app engagement. This is currently exploratory — no production dashboards depend on it yet, and the team is proof-of-concepting what insights are possible.

Table: `events`

One row per event tracked in the NALA mobile app.

Column	Type	Description
event_id	string	Primary key. Unique event identifier assigned by Amplitude.
user_id	string	The NALA user ID (matches `Source 1 users.id`). Null for anonymous events before the user logs in or creates an account.
device_id	string	Unique device identifier. Present for all events including anonymous ones.
event_type	string	The event name, using dot-notation (e.g. `sign_up.started`, `sign_up.completed`, `kyc_step.completed`, `send_money_screen.started`, `transaction.completed`, `open_wallet_account.completed`, `app.opened`).
event_time	timestamp	Timestamp of the event on the client device.
event_properties	json	Event-specific properties. Structure varies by event type.
user_properties	json	User properties at the time of the event. May contain `email`, `account_status`, `plan_type`, `signup_date`.
platform	string	Client platform. Values: `iOS`, `Android`.
os_name	string	Operating system name.
country	string	ISO country code derived from the user's IP address.
city	string	City name derived from IP geolocation.
app_version	string	Version of the NALA app that generated the event.
session_id	bigint	Session identifier grouping events within a single app session.
server_upload_time	timestamp	Timestamp when Amplitude's servers received the event.

Approximate row count: 100,000,000
Update frequency: Daily batch load at approximately 06:00 UTC. Data reflects events from the previous day.

Business Requirements

The transformation layer must serve the following needs. Requirements 1–4 are production-ready and must support operational dashboards and the semantic layer. Requirements 5–6 are exploratory.

Production Requirements

1. Transaction Reporting (Finance)

The finance team needs a daily view of completed transaction volume by currency corridor. This must include:

- Transaction count and total sent amount in both the local sending currency and USD
- Ability to filter by corridor (e.g. GBP-KES, USD-TZS), transaction type, and date
- Only completed transactions that represent real outbound money movement to a recipient (the candidate should determine which transaction types qualify based on the type values provided)

2. Disbursement Operations (Ops)

The operations team monitors payout performance. They need:

- Disbursement attempt success rate by provider
- Average time from attempt creation to completion (and to failure)
- Time-to-complete banded into categories: under 1 minute, 1–5 minutes, 5–30 minutes, 30 minutes–1 hour, 1–24 hours, over 24 hours
- Ability to identify which providers are slow or unreliable

3. Fincrime Operations (Fincrime Ops)

The fincrime team needs visibility into how the rule engine is performing:

- Rule execution volumes by rule name and category
- False positive rate per rule (based on `rule_execution_reviews`)
- Average time from rule execution to review completion
- Ability to track which rules generate the most noise (high trigger rate with high false positive rate)

4. Fincrime Task Pipeline (Fincrime + Backend)

When the fincrime system flags a user or transaction, it creates a task in the backend for manual review. The team needs to understand this pipeline:

- How many fincrime-triggered tasks are created daily
- What proportion are resolved, and what is the average resolution time
- How task outcomes (approved, rejected, escalated) correlate with the original workflow execution result
- This requires joining data across Source 1 and Source 2

Exploratory Requirements

5. Onboarding Funnel (Growth)

The growth team wants to understand user conversion through the onboarding funnel using Amplitude events:

- Signup started → Signup completed → KYC completed → First transaction completed
- Conversion rates between each step
- Average time between steps
- This is a proof-of-concept — the team wants to see what's possible before committing to production dashboards

6. App Engagement (Product)

Product wants to explore which app events and behaviours correlate with transaction completion. This is open-ended — they want the data available for ad-hoc analysis, not a specific dashboard.

Metrics to Define

Using dbt Semantic Layer (MetricFlow) syntax, define the following 7 metrics. Each metric should be backed by a semantic model with appropriate entities, dimensions, and measures.

1. `completed_transaction_volume`

Sum of `sent_amount` for completed, volume-qualifying transactions. Must support slicing by currency corridor, sending currency, and date at a daily grain.

2. `transaction_success_rate`

Ratio of completed transactions to all volume-qualifying transactions (only transaction types that represent outbound money movement to a recipient). Must support slicing by corridor and date.

3. `disbursement_provider_success_rate`

Ratio of completed disbursement attempts to total disbursement attempts. Must support slicing by provider name and date.

4. `fincrime_false_positive_rate`

Ratio of `FALSE_POSITIVE` reviews to total reviews in `rule_execution_reviews`. Must support slicing by rule name, rule category, and date.

5. `signup_to_first_transaction_hours` *(exploratory)*

Average number of hours between a user's `sign_up.completed` event and their first `transaction.completed` event, derived from Amplitude data. Must support slicing by country and date.

6. `daily_signups`

Count of new user registrations per day, based on the `created` timestamp in the backend users table. Must support slicing by sender country and date.

7. `daily_activations`

Count of users who became approved to transact (post-KYC approval) per day, based on the `allowed` timestamp in the backend users table. A user is considered "activated" when their `allowed` timestamp is set. Must support slicing by sender country and date.

What to Submit

Deliverable 1: Architecture Document (2–4 pages)

A written document explaining your approach. It must cover:

- **Snowflake warehouse provisioning** — What databases, schemas, warehouses, and roles would you create? How would you size and configure warehouses for different workloads (dbt transformation vs. BI queries vs. development)?
- **Orchestration plan** — How would you schedule dbt jobs? Which models need to run hourly, daily, or on-demand? How would you handle CI for pull requests and manage the dev-to-production deployment workflow?
- **Layering philosophy** — How did you structure your staging, intermediate, and mart layers? Why?
- **Materialization rationale** — Why did you choose each materialization strategy (view, table, incremental, ephemeral)? Pay particular attention to how you handle the difference between production-ready and exploratory models.
- **CDC handling** — How do you handle StreamServe data in staging? How do you approach deduplication, soft deletes, and incremental loading?
- **Cross-source enrichment** — How do you join fincrime data (Source 2) back to backend data (Source 1) through the `context` JSON column? At which layer does this happen?
- **Testing strategy** — What do you test and at which layer? What is the minimum testing standard for a production mart vs. an exploratory model?
- **Semantic layer approach** — How do you structure your semantic models? Which mart models do you expose, and why?

Deliverable 2: dbt Project Folder

A complete dbt project (does not need to be runnable). It should contain:

- `dbt_project.yml` with appropriate configuration
- Source YAML definitions for all three data sources
- Staging models for all source tables
- Intermediate models where you see fit
- At least 3 mart models covering the production requirements (1–4)
- At least 1 model for the exploratory requirements (5–6)

- Semantic model YAML with the 7 required metrics defined in MetricFlow syntax
- Full YAML documentation on all models: descriptions, column descriptions, data types, and tests
- A clear, consistent folder and file naming structure
- An `AGENTS.md` (or equivalent) at the root of the project — a reference document that instructs AI coding agents on how to work within this repository. It should cover the project's conventions, coding standards, naming patterns, and any rules an agent must follow when generating or modifying code. Think of it as the onboarding guide you would write for both a new team member and an AI assistant.

Deliverable 3: AI Prompt Log

A document containing every prompt you used with AI tools during this exercise. For each prompt, note:

- The tool used (e.g. Cursor, Codex, Claude Code)
- The prompt text
- Whether you accepted, modified, or rejected the output
- Brief notes on any modifications you made

This is not a trick — we use AI tools extensively and expect you to. The log helps us understand how you work with AI as a collaborator. We are looking for evidence that you directed the AI rather than the other way around.

Submission

- **Format:** Zip file containing all three deliverables, or a link to a private GitHub repository.
- **Deadline:** 7 days from receipt.
- **Questions:** If anything in this brief is ambiguous, make a reasonable assumption and state it in your architecture document. In a real project, you would ask — here, we want to see what assumptions you make and how you communicate them.

What Happens Next

After submission, you will be invited to a 60-minute live session with a member of the data team. You will:

1. Walk through your architecture document and explain your decisions
2. Be asked pointed questions about specific models in your project
3. Discuss a live scenario involving a data discrepancy between two dashboards
4. Have time to ask your own questions about the team and the role

The live session is where the real signal comes from. The take-home shows us what you can build; the live session shows us how you think.